

ActiHome: guia tecnica de estudio

Como se construye una aplicacion Java con Swing, Spring Boot, JPA y MySQL

Este documento no explica la app como usuario. Explica la teoria y la forma de pensar que hay detras del codigo, para que puedas entenderlo, defenderlo y continuar desarrollandolo.

Idea central: ActiHome esta hecha por capas: la interfaz recoge acciones, los servicios aplican reglas, los DAOs hablan con la base de datos y las entidades representan los datos.

1. Mapa mental del proyecto

La primera idea importante es que una aplicacion no se construye poniendo todo el codigo junto. Se divide en responsabilidades. En ActiHome cada paquete tiene un papel.

Parte	En el proyecto	Responsabilidad
Arranque	ActihomeApplication	Inicia Spring Boot y abre la primera ventana Swing.
Interfaz	ui	Pantallas, botones, tablas y formularios.
Sesion	SessionManager	Recuerda que usuario ha iniciado sesion.
Servicios	model/services	Reglas de negocio: login, reserva, reviews, alojamientos.
Entidades	model/entities	Clases Java que representan tablas de la base de datos.
DAOs	UserDao, HousingDao...	Consultas y operaciones contra MySQL mediante Spring Data JPA.
Base de datos	schema.sql / data.sql	Estructura de tablas y datos iniciales.
Tests	src/test	Pruebas que validan la logica del proyecto.

2. Arquitectura por capas

La arquitectura por capas evita mezclar codigo de pantalla, reglas de negocio y base de datos. Esto hace que el proyecto sea mas facil de entender, probar y modificar.

```
Usuario pulsa un boton en Swing
|
v
LoginFrame / ReserveHousingFrame / UploadHousingFrame
|
v
UserService / ReservationService / HousingService
|
v
UserDao / ReservationDao / HousingDao
|
v
MySQL
```

Ejemplo: una ventana no deberia comprobar si una contrasena es correcta ni guardar usuarios directamente. La ventana recoge datos y llama al servicio. El servicio decide y usa el DAO.

```
sessionManager.login(userService.login(username, password));
```

3. Spring Boot en una app de escritorio

Normalmente Spring Boot se asocia con aplicaciones web, pero aquí se usa como motor interno para gestionar objetos, servicios, repositorios, transacciones y configuración. La interfaz sigue siendo Swing.

```
SpringApplication app = new SpringApplication(ActihomeApplication.class);
app.setWebApplicationType(WebApplicationType.NONE);
ConfigurableApplicationContext context =
SpringApplication.run(ActihomeApplication.class, args);
```

La clave es `WebApplicationType.NONE`: se le dice a Spring que no levante un servidor web. Después se obtiene una ventana desde el contenedor de Spring.

```
EventQueue.invokeLater(() -> {
LoginFrame loginFrame = context.getBean(LoginFrame.class);
loginFrame.setVisible(true);
});
```

4. Inyección de dependencias

La inyección de dependencias significa que tus clases no crean manualmente todo lo que necesitan. Spring crea los objetos y se los entrega. Esto se ve con `@Autowired` o con parámetros en el constructor.

```
@Autowired
private UserDao userDao;
```

En vez de construir un `UserDao` con `new`, Spring proporciona una implementación automáticamente. Esto permite cambiar piezas, testear mejor y mantener el código más limpio.

Anotación	Significado
<code>@SpringBootApplication</code>	Marca la clase principal de arranque.
<code>@Component</code>	Spring puede crear esta clase como objeto gestionado.
<code>@Service</code>	Clase de lógica de negocio gestionada por Spring.
<code>@Autowired</code>	Spring inyecta una dependencia.
<code>@Transactional</code>	Ejecuta operaciones dentro de una transacción.
<code>@Entity</code>	La clase representa una tabla de base de datos.

5. Entidades: convertir el dominio en clases

Una entidad es una clase que representa un concepto importante de la aplicación y normalmente una tabla de la base de datos. En ActiHome las entidades principales son `User`, `Housing`, `Reservation` y `Review`.

```
@Entity
@Table(name = "HOUSINGS")
public class Housing {
private Long id;
private Long housingCode;
private String type;
private int numberOfRooms;
private BigDecimal pricePerNight;
private User owner;
}
```

La clase `Housing` no es solo una bolsa de datos: es el modelo Java de un alojamiento. JPA se encarga de traducir sus campos a columnas.

Campo Java	Tipo	Idea
id	Long	Identificador interno generado por la base de datos.
housingCode	Long	Codigo visible o de negocio del alojamiento.
pricePerNight	BigDecimal	Precio por noche, usando tipo exacto para dinero.
available	boolean	Indica si se puede reservar.
owner	User	Relacion con el usuario propietario.

6. Relaciones entre entidades

Las relaciones permiten expresar que un objeto depende de otro. Por ejemplo, un alojamiento tiene un propietario, una reserva tiene un cliente y un alojamiento, y una review tiene un autor y un alojamiento.

```
@ManyToOne(optional = false, fetch = FetchType.EAGER)
@JoinColumn(name = "ownerId")
public User getOwner() {
    return owner;
}
```

ManyToOne significa muchos a uno. Muchos alojamientos pueden pertenecer a un mismo usuario. JoinColumn indica la columna de la tabla que guarda esa relacion.

Relacion	Tipo	Explicacion
Housing -> User	ManyToOne	Muchos alojamientos pueden tener el mismo propietario.
Reservation -> User	ManyToOne	Muchas reservas pueden ser del mismo cliente.
Reservation -> Housing	ManyToOne	Muchas reservas historicas pueden referirse a un alojamiento.
Review -> User	ManyToOne	Muchas reviews pueden ser escritas por usuarios.
Review -> Housing	ManyToOne	Un alojamiento puede tener muchas reviews.

7. DAOs y Spring Data JPA

Un DAO o repository es la pieza que permite leer y guardar datos sin escribir todo el SQL manualmente. En tu proyecto extienden PagingAndSortingRepository.

```
public interface UserDao extends PagingAndSortingRepository<User, Long> {
    boolean existsByUsername(String username);
    Optional<User> findByUsername(String username);
}
```

Spring interpreta el nombre de los metodos. findByUsername equivale conceptualmente a buscar en USERS por la columna username. existsByUsername devuelve true o false.

Optional: Optional<User> expresa que puede haber usuario o puede no haberlo. Obliga a comprobar el caso vacio antes de usar el objeto.

8. Servicios: donde vive la logica real

Los servicios son la parte mas importante para entender como se hace la app. Aqui se aplican las reglas: quien puede reservar, cuando se puede hacer check-in, como se calcula una puntuacion, etc.

```
public void signUp(User user) throws DuplicateInstanceException {
    if (userDao.existsByUsername(user.getUsername())) {
        throw new DuplicateInstanceException("project.entities.user", user);
    }

    user.setPassword(passwordEncoder.encode(user.getPassword()));
    userDao.save(user);
}
```

Este metodo registra un usuario. La teoria aplicada es sencilla: validar, transformar si hace falta y guardar. Si algo incumple una regla, se lanza una excepcion.

Servicio	Responsabilidad principal
UserService	Registro, login, actualizar perfil y cambiar contrasena.
HousingService	Crear, listar, filtrar, editar e intercambiar alojamientos.
ReservationService	Reservar alojamiento, ver reservas y hacer check-in.
ReviewService	Publicar, listar y actualizar reviews.
PermissionChecker	Comprobar que usuarios existen y recuperar usuarios validos.

9. Ejemplo completo: login

El login es un buen ejemplo para entender el flujo completo desde la pantalla hasta la base de datos.

```
private void login() {
    String username = usernameField.getText();
    String password = new String(passwordField.getPassword());

    sessionManager.login(userService.login(username, password));
}
```

La ventana recoge el texto del formulario. Despues llama a userService.login. El servicio busca el usuario y valida la contrasena.

```
Optional<User> user = userDao.findByUsername(username);

if (!user.isPresent()) {
    throw new IncorrectLoginException(username, password);
}

if (!passwordEncoder.matches(password, user.get().getPassword())) {
    throw new IncorrectLoginException(username, password);
}

return user.get();
```

El password no se compara directamente con == ni equals. Se usa BCrypt porque la contrasena guardada esta cifrada. matches compara la contrasena introducida con el hash almacenado.

10. Ejemplo completo: reservar alojamiento

La reserva muestra varias reglas de negocio encadenadas. Este es el tipo de metodo que conviene saber explicar en una entrevista.

```

if (customer.getRole() != RoleType.CUSTOMER) {
    throw new NotAuthorizedUserException();
}

if (!housing.get().isAvailable()) {
    throw new AlreadyReservedException();
}

if (checkInDate.isBefore(reservationDate)) {
    throw new MustBeTodayOrAfterException();
}

if (checkOutDate.isBefore(checkInDate.plusDays(1))) {
    throw new CheckOutMustBeOneDayAfterException();
}

```

Fijate en el orden: primero se comprueba el rol, despues que el alojamiento exista y este disponible, despues las fechas y la tarjeta. Solo al final se crea la reserva.

```

long nights = ChronoUnit.DAYS.between(
    checkInDate.toLocalDate(),
    checkOutDate.toLocalDate()
);

BigDecimal totalPrice = housing.get().getPricePerNight()
    .multiply(BigDecimal.valueOf(nights));

Reservation reservation = new Reservation(
    reservationCode,
    checkInDate,
    checkOutDate,
    "Tarjeta de credito",
    reservationDate,
    totalPrice,
    false,
    customer,
    housing.get()
);

housing.get().setAvailable(false);
return reservationDao.save(reservation);

```

Transaccion: Crear la reserva y marcar el alojamiento como no disponible deberia ocurrir como una unica operacion. Para eso sirve `@Transactional`.

11. Excepciones propias

Las excepciones propias convierten errores del negocio en clases con nombre. Esto hace que el codigo se lea mejor y que la interfaz pueda mostrar mensajes adecuados.

Excepcion	Cuando aparece
DuplicateInstanceException	Se intenta registrar algo que ya existe.
IncorrectLoginException	Usuario o contrasena incorrectos.
NotAuthorizedUserException	El rol del usuario no permite esa accion.
AlreadyReservedException	El alojamiento ya no esta disponible.
ScoreOutOfBoundsException	Una puntuacion de review esta fuera de 0 a 5.
CannotCheckInException	Se intenta hacer check-in antes de la fecha.

Una excepcion propia no significa que la aplicacion se haya roto. Muchas veces significa que una regla se ha cumplido correctamente: se ha bloqueado una accion invalida.

12. Swing: como se construyen las pantallas

Swing trabaja con ventanas y componentes: JFrame, JPanel, JButton, JLabel, JTable, JTextField, etc. En tu proyecto cada pantalla suele extender JFrame.

```
public class LoginFrame extends JFrame {
    private JTextField usernameField;
    private JPasswordField passwordField;
}
```

La pantalla se monta creando componentes y asignando eventos. Un evento es una accion del usuario, por ejemplo pulsar un boton.

```
JButton loginButton = new JButton("Iniciar sesion");
loginButton.addActionListener(e -> login());
```

La teoria aqui es programacion orientada a eventos: el programa no avanza de arriba abajo como un script, sino que espera acciones del usuario.

13. Sesion de usuario

SessionManager guarda el usuario que ha iniciado sesion. Es una clase sencilla, pero importante porque permite que las ventanas sepan quien esta usando la app.

```
private User loggedInUser;

public void login(User user) {
    loggedInUser = user;
}

public boolean isUserLoggedIn() {
    return loggedInUser != null;
}
```

Este patron es comun: guardar el estado de sesion en un componente compartido. En una app web esto se haria de otra forma, pero en Swing tiene sentido.

14. Base de datos y SQL

schema.sql define la estructura de la base de datos. data.sql mete datos iniciales. Aunque JPA trabaja con objetos, por debajo todo termina en tablas.

```
CREATE TABLE HOUSINGS (
    id BIGINT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    housingCode BIGINT NOT NULL,
    type VARCHAR(30) NOT NULL,
    numberOfRooms INTEGER NOT NULL,
    pricePerNight DECIMAL(10, 2) NOT NULL,
    ownerId BIGINT NOT NULL,
    CONSTRAINT OwnerIdFK FOREIGN KEY(ownerId)
    REFERENCES Users(id)
);
```

La columna ownerId es una clave externa. Eso significa que un alojamiento no guarda todo el usuario dentro, sino el id del usuario propietario.

15. Tests: aprender el codigo por casos

Los tests explican que espera el proyecto de cada servicio. Son casi una documentacion ejecutable.

```
assertThrows(DuplicateInstanceException.class,
    () -> userService.signUp(user2));
```

Este test dice: si intento registrar dos usuarios con el mismo username, el sistema debe lanzar DuplicateInstanceException. No solo prueba codigo: documenta una regla.

Tipo de test	Que comprueba
Caso correcto	La accion funciona y devuelve lo esperado.
Caso de error	La accion se bloquea con la excepcion correcta.
Orden	Listados de reservas o reviews aparecen en el orden esperado.
Calculos	Precio total o puntuacion media se calculan bien.

Detalle observado: Los reportes guardados indican que algunos tests de HousingService fallaban por orden/datos de prueba mezclados. Es un buen punto de mejora para portfolio.

16. Como se construiria desde cero

Si tuvieras que rehacer ActiHome desde cero, el orden recomendable seria este:

- Definir requisitos: roles, acciones y reglas principales.
- Diseñar el modelo: User, Housing, Reservation y Review.
- Diseñar la base de datos y sus relaciones.
- Crear entidades JPA con @Entity, @Id, @ManyToOne y @JoinColumn.
- Crear DAOs para operaciones basicas y consultas necesarias.
- Crear servicios con interfaces e implementaciones.
- Crear excepciones propias para reglas incumplidas.
- Crear tests de servicios antes de pulir la interfaz.
- Crear ventanas Swing que llamen a los servicios.
- Probar el flujo completo y corregir errores.

17. Conceptos que debes dominar

Concepto	Para que te sirve en ActiHome
POO	Crear clases que representan conceptos reales.
Encapsulacion	Proteger datos mediante atributos privados y getters/setters.
Interfaces	Definir contratos como UserService sin depender de una implementacion concreta.
Spring Boot	Arranque, configuracion e inyeccion de dependencias.
JPA	Mapear objetos Java a tablas SQL.
Repositories	Consultar la base de datos con metodos Java.
Transacciones	Evitar operaciones guardadas a medias.
Excepciones	Representar errores de reglas de negocio.
Swing	Crear interfaz de escritorio basada en eventos.
JUnit	Probar que las reglas funcionan.

18. Mejoras para portfolio

Para convertir el proyecto en algo mas presentable, no hace falta reescribirlo todo. Puedes mejorar por fases.

- Crear un README profesional con capturas, tecnologias y funcionalidades.
- Arreglar los tests que fallan y explicar el aprendizaje.

- Corregir textos con caracteres raros por problemas de codificación.
- Anadir validaciones mas completas en formularios.
- Separar mejor logica de interfaz en algunas ventanas.
- Crear datos de prueba limpios.
- Anadir capturas o un pequeno video de uso.
- Opcional: migrar la interfaz a JavaFX o crear una API REST con frontend web.

19. Frase para defender el proyecto

ActiHome es una aplicacion de escritorio desarrollada en Java con Swing y Spring Boot. Usa una arquitectura por capas, persistencia con JPA sobre MySQL, servicios transaccionales para la logica de negocio, excepciones propias para validar reglas del dominio y tests con JUnit para comprobar los casos principales.

20. Ruta de estudio recomendada

- Leer primero las entidades y dibujar las tablas.
- Leer los DAOs y traducir cada metodo a una consulta SQL mental.
- Leer UserServiceImpl completo hasta poder explicarlo sin mirar.
- Leer ReservationServiceImpl porque concentra muchas reglas reales.
- Leer una ventana Swing y seguir el flujo boton -> servicio -> DAO.
- Leer los tests como si fueran ejemplos de uso.
- Modificar una funcionalidad pequena y crear su test.

Fin de la guia